PCBSmith — Publication Plan

Conference paper topics derived from the prompt-to-PCB pipeline (D:\AI\PCB designer, branch codex/circuit-intelligence-slice). Compiled 2026-07-08, revision 2 (experiment design + rigor requirements).

Strategy in one paragraph

One flagship system paper anchors the portfolio and every other paper cites it. Each additional paper must own a distinct axis (its own experiment, its own related-work neighbourhood) so reviewers never see two submissions telling the same story. Because PCBSchemaGen (arXiv 2602.00510) and Diode Computers have already published or commercialized the propose-verify-refine loop, priority matters: put the flagship on **arXiv as a preprint immediately** to timestamp the ideas that are uniquely ours (the underestimating pre-filter discipline, the failure-driven check ratchet, evidence-backed components, fabrication-ready end-to-end output). The project being unfinished is an advantage: papers 2-5 each add a planned feature and measure it, so research and development are the same work.

Conference vs journal split

Plan: most topics go to conferences/workshops; 1-2 become journal papers. The natural journal candidates are (a) the **flagship system paper expanded** — conference version first, then the journal version adds the full case-study set, the escape-statistics longitudinal analysis, and whatever Papers 2/4/5 produced by then (IEEE Access or ACM TODAES); and (b) the **tool-use study expanded** — the conference version covers PCB + math/CAS, the journal version adds the CAD/3D-print domain and the full cross-model matrix. Journal versions must add >=30% new material over their conference versions; the plan below is structured so that this happens naturally.

Reading the effort estimates

“Writing only” = the data already exists in the repo (outputs/, docs/, tests/golden, commit history). “Experiments” = new runs are needed but the harness exists. “New build” = code must be written first.

Experimental integrity rules (Papers 2, 3, 7)

These rules exist because the comparisons are easy to accidentally invalidate. Write them into the methods section verbatim; they are the first things reviewers probe.

The three-condition design

- **Condition A — pure generation:** the model answers from its own knowledge. No tools, no interpreter. Hallucination is measured here (invented pin functions, wrong component limits, wrong closed forms).
- **Condition B — code-writing allowed:** the model may write and execute its own code in a BARE interpreter (stdlib only). This answers the “it could just write the tool itself” objection honestly instead of dodging it. Expected result: B beats A on accuracy but burns tokens; C beats both.
- **Condition C — curated tools + verifier feedback:** the domain tools (calculators, footprint census, CAS, geometry queries) plus findings-with-positions fed back per iteration.

Isolation rule 1 — condition isolation

Condition B must never see the project's tools, libraries, or any artifact produced under Condition C. Fresh context per attempt; no calculator outputs, footprint tables, or verifier findings leaking into prompts. For Paper 2's ablations the same rule applies to check layers: when a layer is disabled, its findings must not re-enter through another channel (e.g. the layout scorer re-reporting what the disabled virtual-DRC check would have said).

Isolation rule 2 — memorization isolation

The benchmark circuits are famous (the 555 servo tester is a widely-copied internet circuit; frontier models have plausibly seen it in training). A raw-condition model reproducing it correctly proves memorization, not capability. Every task must therefore be PERTURBED: novel value combinations, shifted supply voltages, and requirement tweaks that make the memorized answer measurably wrong. Same discipline for math (no textbook integrals verbatim) and CAD (parametric variations). The oracle grades the perturbed spec, so memorized answers fail. State this in the paper explicitly.

Per-paper rigor: depth, not breadth

Only Paper 3 buys credibility with domain breadth. Every other paper buys it with a specific kind of depth:

- **Paper 1:** archaeological rigor — mine git history for escapes-per-board declining over ten boards, stage timings, iteration counts. The make-or-break is the positioning against PCBSchemaGen/Diode, not new experiments.
- **Paper 2:** ablation isolation (rule 1 above) — a disabled check layer must be genuinely absent from the feedback loop.
- **Paper 4:** formalization — the virtual DRC is internally SEVERAL independent instances of the pattern (copper clearance, courtyards, pour connectivity, silk), each with its own underestimation argument and its own documented escape; present them as multiple instances and connect to the static-analysis soundness literature. No new domain needed.
- **Paper 5:** one external baseline — run FreeRouting (or KiCad's own autorouting path) on the same boards and show it cannot express creepage constraints. Without the baseline it is an experience report; with it, an algorithms paper.
- **Paper 6:** completeness — enumerate EVERY golden catch and EVERY false positive from history, not a curated sample.

Paper 1 — PCBSmith: a Deterministic, Evidence-Backed Pipeline from Prompt to Fabrication-Ready PCB

Type: Flagship system paper. Write FIRST; everything else cites it.

What it is

The end-to-end architecture: intent → topology → datasheet-grounded calculators → composition → KiCad schematic → ERC → ngspice → board → virtual DRC → kicad-cli DRC/parity → design checks → review bundle that is structurally incapable of self-approving. Ten topologies from a voltage divider to a mains-isolated flyback and the automation-routed 555 servo tester, all regenerating live as a regression suite.

Contribution claims

- A complete prompt-to-Gerber pipeline where every fact carries evidence (sha-pinned datasheets, page-level locators) and every design rule is executable ("a rule that is not a check is a wish").
- The two-tier verification split: a deliberately underestimating virtual DRC pre-filter with the real EDA tool as final authority.
- The failure-driven ratchet: every live DRC escape across ten boards became a permanent machine check — quantifiable as escapes-per-board declining over the project history.
- Honest positioning against PCBSchemaGen, Diode, Quilter, atopile, tscircuit, JITX — independent convergence strengthens the architecture claim.

How to do it

1. Reconstruct the timeline and escape statistics from git history and docs/project-history.md (every board's iteration count and which checks were born from which failures).

2. Present 3 case studies: the flyback (safety/creepage), the servo tester (automation-routed, router beat the hand layout 731mm/8 vias vs 754mm/10), and one simple board for contrast.

3. Measure the pipeline: wall-clock per stage, virtual-DRC vs kicad-cli round-trip cost, golden-suite runtime.

4. Write the related-work section from docs/market-notes/ (already researched).

5. Post to arXiv immediately; submit to the next matching CFP.

Supporting parts of the project

- docs/project-history.md and CLAUDE.md (methodology, five laws)
- docs/market-notes/ (related-work survey already done)
- outputs/ — all ten boards with review bundles, DRC reports, fab packages
- tests/golden/test_regenerate_all.py (the 9-board live regression)
- docs/pcb-design-rules.md (the executable rulebook)

Multi-model testing

Not required. The pipeline is deterministic; the LLM is not in the design loop. One paragraph explains where an LLM can optionally sit (topology forge), citing Paper 4.

Venues: MLCAD, DATE, ISQED; next MECO cycle; journal expansion to IEEE Access / ACM TODAES later **Effort:** Writing only — 2-3 weeks

---

Paper 2 — From Limits to Results: Constraint Systems as an Efficiency Mechanism for AI Design Loops

Type: Method paper with ablations (the user's original thesis).

What it is

The claim: hard constraints and machine checks do not restrict an AI design system — they are what make it converge cheaply. Findings with positions act as a reward oracle with error localization; the check stack turns an open-ended generation problem into guided search. Measured by ablation: same design tasks with check layers progressively disabled.

Contribution claims

- Quantified ablation: iterations-to-clean, tokens consumed, and authority-tool invocations with and without each check layer (virtual DRC, design checks, card contracts, pre-gates).
- Error localization matters: findings with positions vs binary pass/fail as feedback — convergence rate comparison (the topology forge supports both feedback modes trivially).
- The economics: a 0.01-0.1s pre-filter in front of a minutes-long EDA round trip changes the viable search-loop design entirely.

How to do it

1. Define 5-10 design tasks of graded difficulty from existing topologies (regenerate variants: changed values, changed board sizes, perturbed placements).

2. Build the ablation switch: a DesignChecksSpec/virtual-DRC config that disables named check layers (small code change).

3. Run the matrix: task × ablation level × (optionally) 2 models for the LLM-in-the-loop variant via forge-topology.

4. Report iterations, wall-clock, token counts, and which classes of defect reach kicad-cli in each configuration.

Supporting parts of the project

- kicad/virtual_drc.py, kicad/design_checks.py (the check stack)
- ai/topology_forge.py (propose-verify-refine harness with findings-fed-back — the ablation testbed for the LLM case)
- kicad/layout_score.py + placement_search.py (pre-gates as constraints on search)
- Live examples: the scorer catching the router's creepage violation; placement pre-gates rejecting blind nudges

Multi-model testing

Optional but recommended: 1-2 models (one frontier API model, one local GGUF) for the LLM-in-the-loop ablations; the deterministic-pipeline ablations need none.

Venues: MLCAD, NeurIPS/ICML workshops on LLM agents or verifiers, LLM4Code-style venues **Effort:** Experiments — 3-5 weeks (harness exists, ablation switch is new)

Paper 3 — Tool-Using AI under Verifiable Outcomes: Measuring Token Cost, Accuracy, and Hallucination with a Hard Oracle

Type: Benchmark/evaluation paper (the user's topic 1).

What it is

Most tool-use evaluations rely on fuzzy LLM judges. PCB design gives a binary, industrial-grade oracle: kicad-cli ERC/DRC, ngspice measurements, and datasheet facts either check out or they do not. The paper measures multiple models across the three conditions (pure generation / bare-interpreter code-writing / curated tools + verifier feedback — see the integrity-rules page) and reports token use, correctness, and hallucination rates against that oracle. Conference version: PCB (deep) + math-via-CAS (breadth). Verilog results (VerilogEval etc.) are CITED as external corroboration, not rebuilt. The mechanical-CAD domain is deliberately held out as its own paper (Paper 7) and joins this line only in the journal expansion.

Contribution claims

- A tool-grounding benchmark where correctness is machine-checkable end to end — no human or LLM judging; oracle independent of the tools under test (kicad-cli/ngspice judge; calculators and census are what the model uses).
- The three-condition result: code-writing beats pure generation on accuracy but burns tokens; curated tools + verifier beat both — the quantitative bridge to Paper 2's thesis.
- Cross-model comparison (frontier APIs vs open weights) of tokens-to-correct-answer and hallucinated component facts.
- A reusable public harness: the topology-spec verifier as the scoring oracle, ported to a second domain (CAS) in few lines — the port cost is itself a reported result.

How to do it

1. Freeze a task set: ~20 topology-spec tasks (existing 10 boards + PERTURBATIONS per isolation rule 2 — memorized internet circuits must fail the perturbed spec) plus a math set: SymPy as the tool, INDEPENDENT numerical evaluation at random points as the oracle (never grade with the same CAS call the tool condition used).

2. Implement the three conditions with isolation rule 1 (bare interpreter for condition B; fresh context per attempt).

3. Run 4-6 models: 2-3 paid APIs, 2-3 local GGUFs (Gemma/Qwen already on disk; PCBSchemaGen reports 81.3% with Gemma-4-31B — a replication anchor). GGUFs are 27-32GB CPU-only on the current machine; budget a GPU rental for the matrix.

4. Report accuracy, tokens, iterations, and an error taxonomy per condition x model x domain; release the harness.

Supporting parts of the project

- ai/topology_forge.py + openai_compatible_client (multi-endpoint harness already works)
- components.py card validator (hallucination detector for pin facts)
- calculators/ (ground-truth design equations, all hand-checked in tests)
- ai_assets/models (local GGUFs) + evidence/llm.py (Anthropic + OpenAI-compatible clients already written)

Multi-model testing

Required — this is the paper's core. Minimum 4 models (2 paid, 2 open) for credible claims; budget API costs and local compute time (27-32GB GGUFs run CPU-only at ~10-20 min/attempt on the current machine — consider a GPU rental for the matrix).

Venues: LLM agent/tool-use workshops (NeurIPS/ACL), LLM4Code, or an EDA+ML venue if framed around the EDA oracle **Effort:** Experiments + harness extension — 5-8 weeks + compute budget

---

Paper 4 — Sound-by-Construction Pre-Filters for Expensive Verifiers

Type: Software-engineering/verification method paper — broadest audience outside EDA.

What it is

The design discipline behind the virtual DRC: a fast approximate checker built to UNDERESTIMATE (stadium pad models, exact convex courtyard hulls, honest text boxes) so it can never reject a design the authoritative tool would accept — false positives are structurally impossible, misses are caught by the authority and patched into the consumer (the router's obstacle model), never into the filter. Includes the documented counter-cases: rect-pad corners and duplicate pad numbers escaping to kicad-cli, and how the fix preserved the soundness invariant.

Contribution claims

- A named, transferable pattern (sound pre-filter + authority oracle + consumer-side inflation) with invariants and proofs of the underestimation property for each geometric model.
- Empirical payoff: pre-filter at 0.01-0.1s vs kicad-cli round trips at minutes; fraction of iterations that never touch the authority across the project history.
- The escape catalogue: every divergence between filter and authority ever observed, and the repair taxonomy (fix the consumer, never break the filter's soundness).

How to do it

1. Formalize the stadium/hull models and prove (or argue rigorously) the underestimation property per check.

2. Mine git history for every escape (each is a commit with a test): rect pads, pad twins, silk metrics, fp_circle parsing, bbox-vs-hull.

3. Measure filter precision/recall against kicad-cli over all golden boards + deliberately poisoned variants (the test suite already contains poisoned fixtures).

Supporting parts of the project

- kicad/virtual_drc.py (the filter) + kicad/library.py (exact hull measurement)
- The escape commits: 9267dff (rect pads, pad connectivity), the hull fixes, silk checks — each with before/after DRC output
- tests/unit/kicad/test_virtual_drc.py + golden suite (ground truth generator)

Multi-model testing

Not required. No LLM involved anywhere in this paper.

Venues: Software engineering / testing venues (ISSTA, ICST short papers), or EDA venues as a verification-methodology paper **Effort:** Writing + measurement scripts — 2-4 weeks

---

Paper 5 — Safety-Constraint-Aware Autorouting: Creepage and Isolation as First-Class Routing Constraints

Type: Focused algorithm paper (small, sharp, novel).

What it is

Academic grid routers optimize length and vias; they do not know about mains isolation. This router threads IEC-style creepage requirements (rule 10.1/10.4) into the obstacle model as layer-agnostic keepout groups, covers rect-pad corners its stadium model would underestimate, applies a turn penalty for manufacturing-conventional 45° output, and is judged by a scorecard that re-checks the full rule set — which caught the router's first clearance-legal but creepage-illegal route.

Contribution claims

- Creepage/isolation-aware A* formulation (both-layer keepouts, straddle-part exemptions) — rare in the autorouting literature.
- The propose-verify split for routers: scorer-as-oracle catching constraint classes the router's own model missed.
- Head-to-head vs hand routing on a real mains flyback: full board from bare placements in seconds, beating the human layout on length and via count while honouring the barrier.
- Output quality engineering: diagonals + turn penalty + collinear merge → KiCad-native track quality (264→118 segments, micro-slivers 67→5 on the servo board).

How to do it

1. Benchmark on the existing boards: flyback (isolation), servo (automation-first), plus placement_search candidates as a workload generator.

2. Compare against at least one baseline (e.g. FreeRouting) on completion, length, vias, and — the point — creepage violations the baseline produces because it cannot express the constraint.
3. Ablate: no keepout groups / no rect-pad cover / no turn penalty, showing what each part prevents.

Supporting parts of the project

- kicad/astar_router.py (the whole algorithm incl. this week's output-quality work) + clearance_groups_from_spec
- kicad/layout_score.py + placement_search.py (oracle + workload)
- The flyback and servo boards + their hand-routed predecessors (in git) as baselines

Multi-model testing

Not required. Pure algorithms paper.

Venues: MLCAD, ISPD (physical design — the exact community), DATE **Effort:** Experiments (baseline comparison is the new work) — 3-4 weeks

---

Paper 6 — Golden Regression for Generative Design Systems

Type: Short paper / experience report (cheapest of all).

What it is

Every code change must regenerate nine complete boards live through ERC, simulation, and kicad-cli DRC before it lands. The suite has documented catches on both sides: real regressions stopped, and false positives in the checking models themselves exposed (bbox vs hull on rounded courtyards, keepout vs legal pour). Regression testing for generators — where the artifact is a design, not a program output — is barely covered in the literature.

Contribution claims

- The pattern: full live regeneration as CI for a generative system, with the external tool as the assertion.
- Catch statistics from history: what a golden suite catches that unit tests structurally cannot.
- Cost analysis: ~15 min for nine boards — when it is worth gating on and how to tier it (unit → virtual → golden).

How to do it

1. Mine git/docs history for every golden catch (each is documented) and classify them.
2. Measure suite runtime growth per topology; discuss the opt-in gating discipline (PCBSMITH_GOLDEN=1 before kicad/ or calculator commits).

Supporting parts of the project

- tests/golden/test_regenerate_all.py
- docs/project-history.md catch records; CLAUDE.md discipline sections

Multi-model testing

Not required.

Venues: ICST/ISSTA short papers, testing workshops; pairs well as a companion to Paper 4 **Effort:** Writing only — 1-2 weeks

---

Paper 7 — Hard-Oracle Design Tasks in Mechanical CAD / 3D Printing

Type: Standalone conference paper — the PCBSmith pattern replicated in a second fabrication domain. Start AFTER PCBSmith reaches a satisfactory level.

What it is

The LLM (or a deterministic pipeline, mirroring PCBSmith's architecture) generates OpenSCAD parts against dimensional and printability requirements. Tools = geometry-kernel queries (volumes, clearances, wall measurements); oracle = independent mesh analysis plus a slicer CLI (e.g. PrusaSlicer) checking manifoldness, minimum wall thickness, overhangs, and printability — fabrication-grade software as the judge, exactly like kicad-cli for boards. Nobody has published this pairing. As a separate paper it earns its own venue (CAD/manufacturing or ML-for-engineering) and later merges into the tool-use journal version as the third domain.

Contribution claims

- First hard-oracle benchmark for generative mechanical design judged by slicer/mesh software rather than human or LLM review.
- Direct architectural transfer: the propose-verify-refine loop, the underestimating pre-filter idea (fast mesh heuristics vs authoritative slicer), and the failure-ratchet, replicated outside EDA — evidence the PCBSmith patterns are general.
- Optionally: physical validation — print a sample of accepted parts and measure them (a table of calipered dimensions vs spec is worth a thousand simulated checks to reviewers).

How to do it

1. Define a task family: brackets, enclosures, adapters with parametric dimensional specs and printability constraints (perturbed per isolation rule 2).

2. Build the verifier: trimesh/admesh for mesh soundness + PrusaSlicer CLI for printability; geometry-query tools for the model to use (kept separate from the oracle).

3. Port the forge harness (spec format + verifier — the port-cost measurement carries over from Paper 3).

4. Run the three conditions; if budget allows, print and measure 5-10 accepted parts.

Supporting parts of the project

- ai/topology_forge.py (the domain-agnostic propose-verify-refine shape)
- The PCBSmith architecture as the design template (Paper 1 becomes the citation anchor)
- CLAUDE.md five laws — the transfer checklist for the new domain

Multi-model testing

Required if framed as a tool-use study (same 4-6 model matrix as Paper 3); **not required** if framed as a deterministic-pipeline system paper first (PCBSmith-for-CAD), with the model study as follow-up. The second framing is cheaper and matches how PCBSmith itself was built.

Venues: CAD/manufacturing venues (CAD Conference, ASME IDETC/CIE), ML-for-engineering workshops; feeds the tool-use journal version **Effort:** New build — the largest single-paper effort after Paper 3; months, but reuses every architectural lesson

---

Sequencing and dependencies

#	Paper	Multi-model?	Target	When
1	Flagship system paper	No	Conference, then JOURNAL	Now → arXiv immediately
6	Golden regression (short)	No	Conference short	Parallel with 1
4	Sound pre-filters	No	Conference	After 1 (cites it)
2	From limits to results	Optional (1-2)	Conference	After ablation switch built
5	Safety-aware autorouting	No	Conference	After FreeRouting baseline

3	Tool-use under hard oracle	REQUIRED (4-6)	Conference, then JOURNAL	Needs compute budget
7	CAD/3D-print hard oracle	See topic page	Conference; feeds 3's journal ver.	After PCBSmith satisfactory

Reserve topics (fold in as sections, or spin out later if a paper needs a companion)

- **Simulation as acceptance test** — ngspice .meas criteria as unit tests for generated hardware (metal detector oscillator within 0.1% of the analytic value; detuning demonstrates the detection mechanism in-pipeline). Natural section of Paper 1 or a standalone short paper.
- **Evidence-grounded component models** — sha-pinned datasheets, page-level locators, pin contracts as machine checks; anti-hallucination by provenance. Section of Paper 1; standalone for a trustworthy-AI venue if Paper 3's results make it topical.
- **Human feedback without ML** — board-diff on hand edits, review comments as findings, this week's user catches (silk metrics, trace segmentation, human-readable schematics) each becoming checks or tracked rules. HCI/SE workshop material once a few more cycles accumulate.
- **One netlist, two drawings** — the Track 9.1 machine vs human schematic duality with netlist-equality as the correctness gate. Waits for the renderer to be built.